

C++

Wednesday 17 September 2025 10:22

PDF documenting my iterative process

04/02

Wednesday 4 February 2026 11:20

A* algorithm is a search algorithm used in pathfinding solutions.

[A* Search Algorithm - GeeksforGeeks](#)

[Introduction to A*](#)

[A Systematic Literature Review of A* Pathfinding - ScienceDirect](#)

[What is the A* Search Algorithm?](#)

Why is A* Efficient?

A* combines features of [Dijkstra's algorithm](#) and [Greedy Best-First Search](#). While Dijkstra finds the shortest path by exploring all routes, it can be slow. [Greedy Best-First](#) is faster but may not find the best solution.

A* strikes a balance by considering both the actual and estimated distance, making it more efficient. This approach is ideal for complex environments, where unnecessary explorations can be avoided.

A* uses a heuristic which estimates the distance from a node to a goal. Dijkstra however explores all possible directions from the starting node.

[A* algorithm and Its Heuristic Search Strategy in Artificial Intelligence - GeeksforGeeks](#)

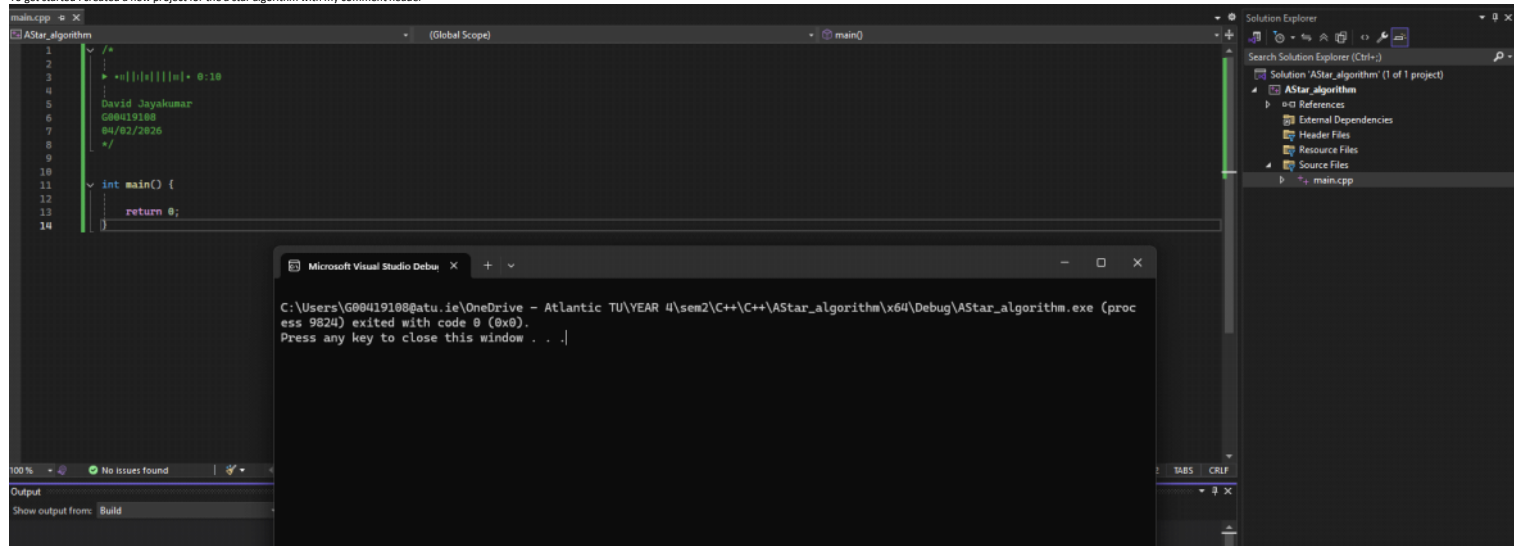
Make a grid
Matrix

```
Std::vector<std::vector<int>> v = {{ 0, 0, 0, 0,},  
                                     {  
                                     }  
                                     }
```

```
0 0 0 0  
0 1 0 0  
0 1 1 0  
0 0 0 1  
1 1 1 0
```

Use a vector to make the grid

To get started I created a new project for the star algorithm with my comment header



[2D Vector in C++ - GeeksforGeeks](#)

I am gonna start by creating a vector of size 5 x 5

Create a 2D vector with fixed number of rows and columns, and initialize all elements to a default value.

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // 3 rows, 4 columns, all elements initialized to 0
    vector<vector<int>> vec(3, vector<int>(4, 0));

    // Modifying elements
    vec[0][0] = 5;
    vec[2][3] = 10;

    for (int i = 0; i < vec.size(); i++)
    {
        for (int j = 0; j < vec[i].size(); j++)
        {
            cout << vec[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

I am gonna hardcode this, I want to implement a way to randomly create the obstacles in a vector. I am gonna need to make a class in the future but for now I shall do it this way

Step-by-Step Implementation Guide

Let's walk through the essential operations you'll need when working with 2D vectors:

Declaration and Initialization

```
#include <vector>
#include <iostream>

int main() {
    // Method 1: Initialize with specific dimensions
    int rows = 5, cols = 4;
    std::vector<std::vector<int>> matrix1(rows, std::vector<int>(cols, 0));

    // Method 2: Initialize with values
    std::vector<std::vector<int>> matrix2 = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    // Method 3: Empty initialization and manual sizing
    std::vector<std::vector<int>> matrix3;
    matrix3.resize(3);
    for(int i = 0; i < 3; i++) {
        matrix3[i].resize(4, -1); // Initialize with -1
    }

    return 0;
}
```

By mixing both those code things, I have this going now

```
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29

#include <iostream> // for input and output
#include <vector> // for using vector container

int main() {
    int rows = 4, cols = 4;

    std::vector<std::vector<int>> matrix(rows, std::vector<int>(cols));

    for(int i = 0; i < rows; ++i) {
        for(int j = 0; j < cols; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << std::endl;
    }

    return 0;
}
```

Microsoft Visual Studio Debu x + -

0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0

C:\Users\G00419188@atu.ie\OneDrive - Atlantic TU\YEAR 4\sem2\C++\AStar_algorithm\Debug\AStar_algorithm.exe (process 26868) exited with code 0 (0x0).
Press any key to close this window . . .

/ Design for Reuse

Minimise hardcoding. Make your code extensible for different grid sizes, heuristics, or weighted graphs.

/ Testing & Validation

I had that for loop and michelle said to change to a for range loop whatever that means, and she advised me to use all to do it

how to do this using a for range loop

```
for(int i = 0; i < rows; ++i) {
    for(int j = 0; j < cols; ++j) {
        std::cout << matrix[i][j] << " ";
    }
    std::cout << std::endl;
}
```

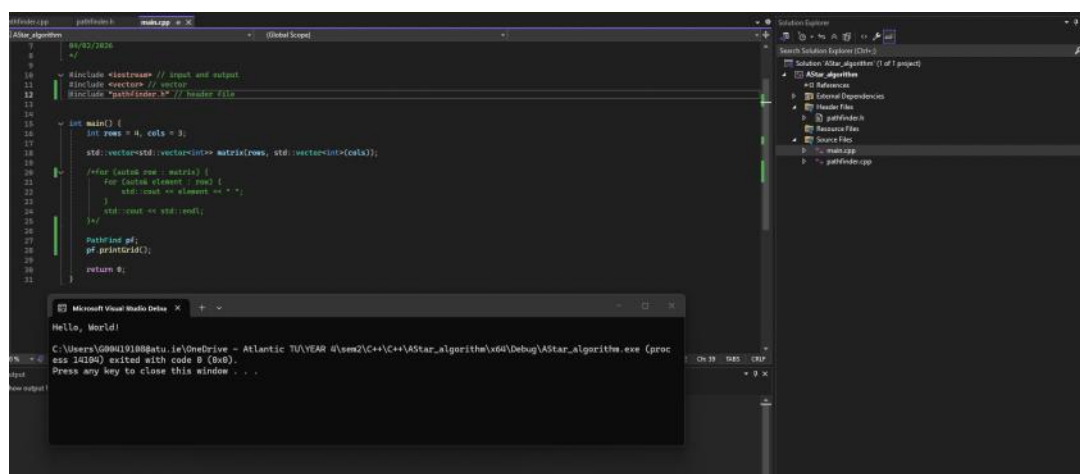
You can use range-based for loops like this:

```
cpp
for(const auto& row : matrix) {
    for(const auto& element : row) {
        std::cout << element << " ";
    }
    std::cout << std::endl;
}
```

[JDSherbett/A-Star-Pathfinding: Simple C++ implementation of an A* Pathfinding class.](#)

Idk where to go from where I am so I am gonna follow their structure and create files like them

Creating the multi file oop project structure



```

1  /*
2  * *|||*|||*|||* 0:10
3  *
4  * David Jayakumar
5  * G00419108
6  * 04/02/2026
7  */
8
9
10 #include <iostream>
11 #include "pathfinder.h"
12
13 void PathFind::printGrid() {
14     std::cout << "Hello, World!" << std::endl;
15 }

```

```

10 #pragma once
11
12 #ifndef PATH_FIND_H
13 #define PATH_FIND_H
14
15 #include <vector> // vector
16
17
18 class PathFind {
19 public:
20     void printGrid();
21 };
22
23 #endif
24
25
26

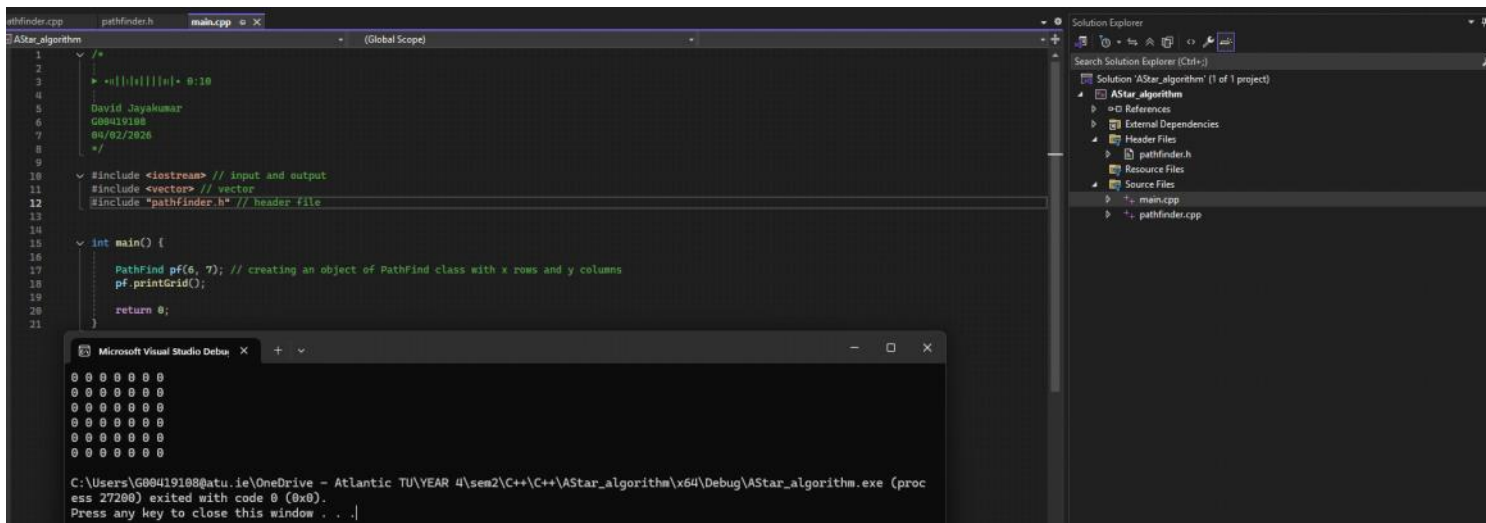
```

Working simple hello world

Now trying to move my code to this multi file structure. I need

[16.2 — Introduction to std::vector and list constructors — Learn C++](#)

[14.10 — Constructor member initializer lists — Learn C++](#)



```

7  04/02/2026
8  */
9
10 #include <iostream>
11 #include "pathfinder.h"
12
13 PathFind::PathFind(int row, int col) : numRows(row), numCols(col) { // constructor to initialize rows and columns
14     grid.resize(numRows, std::vector<int>(numCols, 0)); //initialised to 0, creating a grid of given rows and columns
15 }
16
17 void PathFind::printGrid() {
18     for (auto& row : grid) { //for range loop to print each element in the grid
19         for (auto& element : row) {
20             std::cout << element << " ";
21         }
22         std::cout << std::endl;
23     }
24 }

```

```

10 #pragma once
11
12 #ifndef PATH_FIND_H
13 #define PATH_FIND_H
14
15 #include <vector> // vector
16
17
18 class PathFind {
19 public:
20     PathFind(int row, int col); //constructor to take in rows and columns parameters
21     void printGrid(); //function to print the grid
22 private:
23     std::vector<std::vector<int>> grid; //2D vector to represent the grid
24     int numRows;
25     int numCols;
26 };
27
28 #endif
29
30
31

```

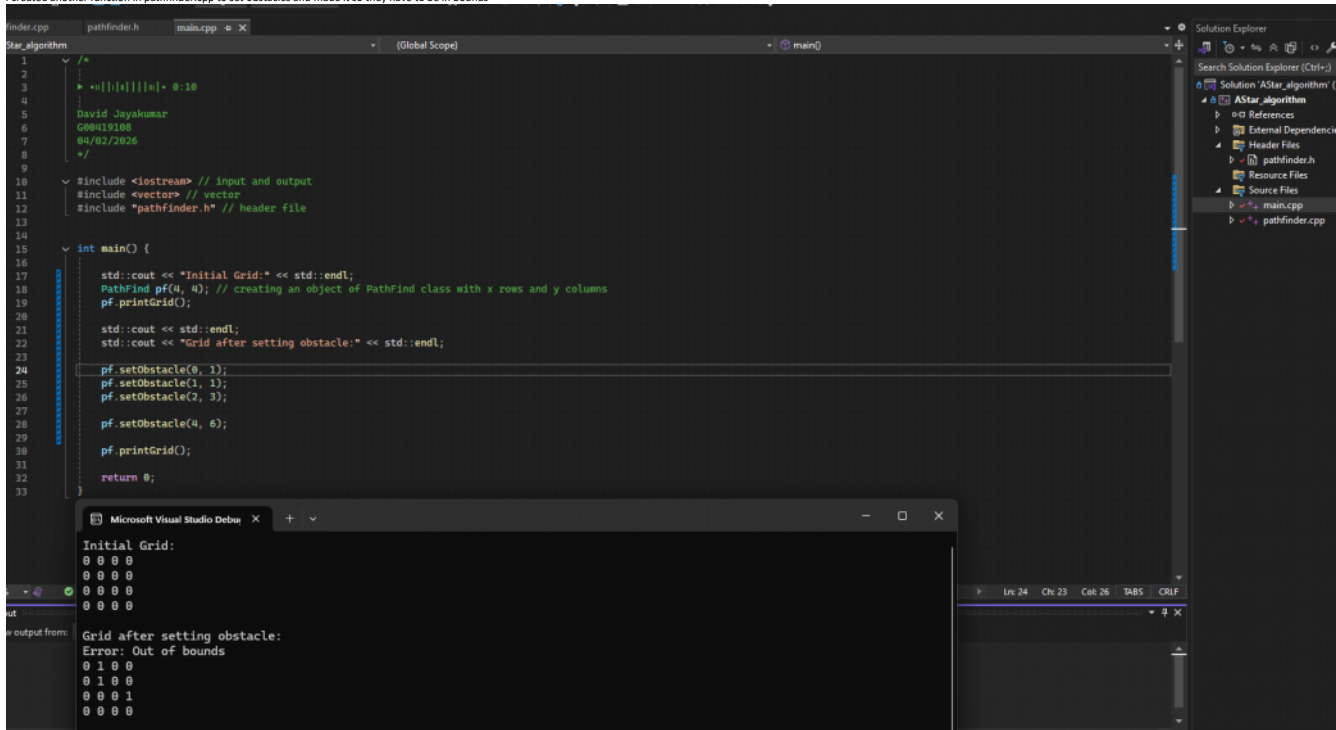
I want to add in the obstacles tho, I don't like how she did it hardcoding everything->
Std::vector<std::vector<int>> v = {{0, 0, 0, 0, 0, 0, 0},

```

    {}
}

```

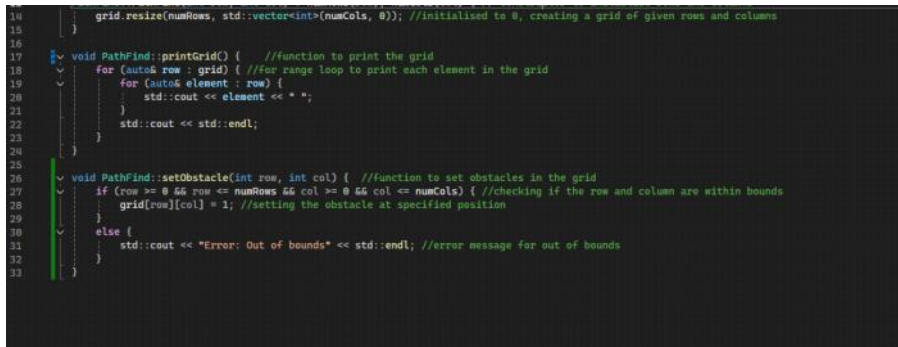
I created another function in pathfinder.cpp to set obstacles and made it so they have to be in bounds



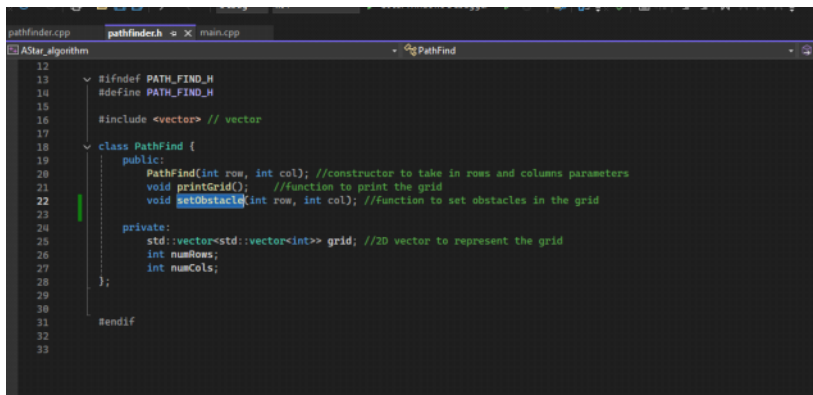
```
1  /*
2  3  4  //||||||n| 0:10
5  David Jayakumar
6  G00419108
7  04/02/2026
8  */
9
10 #include <iostream> // input and output
11 #include <vector> // vector
12 #include "pathfinder.h" // header file
13
14
15
16
17 int main() {
18     std::cout << "Initial Grid:" << std::endl;
19     PathFind pf(4, 4); // creating an object of PathFind class with x rows and y columns
20     pf.printGrid();
21
22     std::cout << std::endl;
23     std::cout << "Grid after setting obstacle:" << std::endl;
24
25     pf.setObstacle(0, 1);
26     pf.setObstacle(1, 1);
27     pf.setObstacle(2, 3);
28
29     pf.setObstacle(4, 6);
30     pf.printGrid();
31
32     return 0;
33 }
```

Initial Grid:
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0

Grid after setting obstacle:
Error: Out of bounds
0 1 0 0
0 1 0 0
0 0 0 1
0 0 0 0



```
14 grid.resize(numRows, std::vector<int>(numCols, 0)); //initialised to 0, creating a grid of given rows and columns
15 }
16
17 void PathFind::printGrid() { //function to print the grid
18     for (auto& row : grid) { //for range loop to print each element in the grid
19         for (auto& element : row) {
20             std::cout << element << " ";
21         }
22         std::cout << std::endl;
23     }
24 }
25
26 void PathFind::setObstacle(int row, int col) { //function to set obstacles in the grid
27     if (row >= 0 && row <= numRows && col >= 0 && col <= numCols) { //checking if the row and column are within bounds
28         grid[row][col] = 1; //setting the obstacle at specified position
29     }
30     else {
31         std::cout << "Error: Out of bounds" << std::endl; //error message for out of bounds
32     }
33 }
```



```
12 #ifndef PATH_FIND_H
13 #define PATH_FIND_H
14
15 #include <vector> // vector
16
17
18 class PathFind {
19 public:
20     PathFind(int row, int col); //constructor to take in rows and columns parameters
21     void printGrid(); //function to print the grid
22     void setObstacle(int row, int col); //function to set obstacles in the grid
23
24 private:
25     std::vector<std::vector<int>> grid; //2D vector to represent the grid
26     int numRows;
27     int numCols;
28 };
29
30
31 #endif
32
33
```

Now that we can add obstacles, I need to add a start and end position but I wanna clean up output because one and zero look funny

For next class, I plan to setup github pages and finish project

[DavidJ7705/AStar_CPluPlu](#)
[AStar_CPluPlu](#)

DavidJ7705 / AStar_CPluPlu

Type [Z] to search

<> Code Issues Pull requests Actions Projects Wiki Security Insights S

AStar_CPluPlu Public

Pin Watch 0

main 1 Branch 0 Tags

Go to file t + Code

DavidJ7705 Initial commit ✓ 9d9a147 · 2 hours ago 1 Commit

README.md Initial commit 2 hours ago

README

AStar_CPluPlu

Abc

No

Rel

No n

Crea

Pac

No p

Publ

https://davidj7705.github.io/AStar_CPluPlu/

Kidding
To implement algorithm - [A Star \(A*\) Path Finding C++ - DEV Community](#)

11/02

Wednesday 11 February 2026 11:01

[C++ Core Guidelines](#)

[Google C++ Style Guide](#)

[Google C++ Style Guide](#)

In: Introduction

P: Philosophy

I: Interfaces

F: Functions

Made it so you can now have a start and goal position on the grid

```
pathfinder.cpp  pathfinder.h  main.cpp  -  X
AStar_algorithm  (Global Scope)
1  /*
2
3  > •|||•|||• 0:10
4
5  David Jayakumar
6  G00419108
7  04/02/2026
8  */
9
10 #include <iostream> // input and output
11 #include <vector> // vector
12 #include "pathfinder.h" // header file
13
14
15 int main() {
16
17     std::cout << "Initial Grid:" << std::endl;
18     PathFind pf(6, 6); // creating an object of PathFind class with x rows and y columns
19     pf.setStart(0, 0); // setting the start point at (0, 0)
20     pf.setGoal(5, 5); // setting the goal point at (5, 5)
21
22     pf.printGrid();
23
24     std::cout << std::endl;
25     std::cout << "Grid after setting obstacle:" << std::endl;
26
27     pf.setObstacle(1, 0);
28     pf.setObstacle(2, 3);
29     pf.setObstacle(3, 0);
30
31
32
33     pf.printGrid();
34
35     return 0;
36 }
```

```
void PathFind::setStart(int row, int col) { //function to set the start point in the grid
    if (row >= 0 && row < numRows && col >= 0 && col < numCols) { //checking if the row and column are within bounds
        startRow = row; //setting the start row
        startCol = col; //setting the start column
    }
    else {
        std::cout << "Error: Start Out of bounds" << std::endl;
    }
}

void PathFind::setGoal(int row, int col) { //function to set the end point in the grid
    if (row >= 0 && row < numRows && col >= 0 && col < numCols) { //checking if the row and column are within bounds
        goalRow = row; //setting the goal row
        goalCol = col; //setting the goal column
    }
    else {
        std::cout << "Error: Goal Out of bounds" << std::endl;
    }
}
```

```

class PathFind {
public:
    PathFind(int row, int col); //constructor to take in rows and columns parameters
    void printGrid(); //function to print the grid
    void setObstacle(int row, int col); //function to set obstacles in the grid
    void setStart(int row, int col); //function to set the start point in the grid
    void setGoal(int row, int col); //function to set the end point in the grid

private:
    std::vector<std::vector<int>> grid; //2D vector to represent the grid
    int numRows;
    int numCols;
    int startRow, startCol;
    int goalRow, goalCol;
};

#endif

```

Initial Grid:

```

S . . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . . G

```

Grid after setting obstacle:

```

S . . . . .
# . . . . .
. . . # . .
# . . . . .
. . . . .
. . . . . G

```

I wanna do it now so that it will randomly set up the grid and stuff but I am not sure how
I also need to go into detail on the manhatan stuff

[A* Algorithm Heuristics for Multiple Goals: Using Manhattan Distance to Find the Nearest Goal in Grid Pathfinding — codegenes.net](#)

Wednesday 18 February 2026 10:58

```
Where I am now, I have the grid now
```

```
Initial Grid:  
S . . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
G
```

```
Grid after setting obstacle:  
S . . . . .  
# . . # . .  
# . . . .  
. . . . .  
G
```

[illegible]

My next logical step is to go from the grid I have to implementing what Michele gave us which is the manhattan distance heuristic and structs

Common Heuristics for Grids:

- **Euclidean Distance:** $(\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2})$. Works for continuous spaces but overestimates in 4-directional grids (no diagonal movement).
- **Chebyshev Distance:** $(\max(|x_1 - x_2|, |y_1 - y_2|))$. Ideal for 8-directional grids (diagonal movement allowed).
- **Manhattan Distance:** $(|x_1 - x_2| + |y_1 - y_2|)$. The focus of this blog—perfect for 4-directional grids (up, down, left, right).

So @ S we must expand and add to the open set, then get the surrounding moves, Calculate the costs of distances to the neighbours and stuff

Step 1: Initialize Open List

Start with (S = (1,1)).

- ($g(S) = 0$) (distance from start to itself).
- ($h(S) = \min(\text{Manhattan}(S, G1), \text{Manhattan}(S, G2))$):
 - ($\text{Manhattan}(S, G1) = |1-3| + |1-4| = 2 + 3 = 5$)
 - ($\text{Manhattan}(S, G2) = |1-5| + |1-2| = 4 + 1 = 5$)
 - ($h(S) = \min(5, 5) = 5$)
- ($f(S) = g(S) + h(S) = 0 + 5 = 5$)
- Open List: [(1,1, f=5)]

I added the Struct Point from the hand written

```
struct Point {
    int row;
    int col;
};
```

The point of Point is that its just a simple coordinate struct which holds the row and col to represent a position on the grid

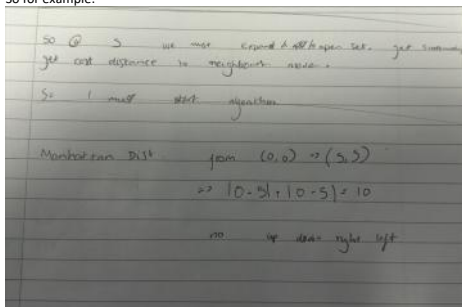
Then added manhattanDistance

```
int manhattanDistance(Point p1, Point p2) {
```

```
int PathFind::manhattanDistance(Point p1, Point p2) {
    return std::abs(p1.row - p2.row) + std::abs(p1.col - p2.col);
}
```

This takes 2 points and calculates how far apart they are by adding the difference in rows and cols

So for example:



From <<https://teams.microsoft.com/v2/>>

This is the $h(n)$ the heuristic cost from n to the goal

This is what A* needs to work correctly as it will never overestimate the distance because it's impossible to get there in fewer moves on the grid

This is known as **admissibility** -> [Admissibility of A* Algorithm - GeeksforGeeks](#)

Admissibility is when an algorithm is guaranteed to find the optimal solution when one exists

Created struct Node

Found this

[JDSherbert/A-Star-Pathfinding: Simple C++ implementation of an A* Pathfinding class.](#)

```
Node::Node(int _x, int _y)
```

```
    : x(_x)
    , y(_y)
    , f(0)
    , g(0)
    , h(0)
{
}
```

So x and y is the row and col

Then we have f is total cost ($g + h$)

G is the cost from the start to the current node

H is the heuristic cost from the current node to the goal

```
struct Node {
    Point point;
    int g; // Cost from start to current node
    int h; // Heuristic cost from current node to goal
    int f; // Total cost (g + h)
};
```

I used Point point which stores the position of this node (row and col) on the grid using the struct

So instead of reinitialising the row and col we use the existing one in Point

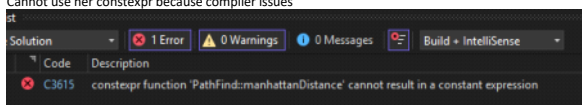
```
struct Node {
    Point point;
    int g; // Cost from start to current node
    int h; // Heuristic cost from current node to goal
    int f; // Total cost (g + h)

    Node* parent; // Pointer to parent node for path reconstruction
    Node(int r, int c):g(0), h(0), parent(nullptr) {
        point = { r, c };
    }
};
```

Node* parent is needed because when we build the Node struct it has to be declared because it reconstructs the path

The constructor is needed as its how nodes are created during the search.

Cannot use her constexpr because compiler issues



```
#include <vector> // vector
#include "pathfinder.h" // header file

int main() {

    std::cout << "Initial Grid:" << std::endl;
    PathFind pf(6, 6); // creating an object
    pf.setStart(0, 0); // setting the start
    pf.setGoal(5, 5); // setting the goal

    pf.printGrid();

    std::cout << std::endl;
    std::cout << "Grid after setting obstacle" << std::endl;

    pf.setObstacle(0, 2);
    pf.setObstacle(1, 2);
    pf.setObstacle(2, 2);
    pf.setObstacle(3, 2);
    pf.setObstacle(4, 2);

    findPath called, start node f cost: 10
```

```
void PathFind::findPath() {
    std::vector<Node> openList; // List of nodes to be evaluated

    std::vector<std::vector<int>> closedList;
    closedList.resize(numRows, std::vector<int>(numCols, 0)); // List of nodes already evaluated

    // Initialize the start node and add it to the open list
    Node startNode(startRow, startCol);
    startNode.h = manhattanDistance(startNode.point, { goalRow, goalCol });
    startNode.f = startNode.g + startNode.h;
    openList.push_back(startNode);

    std::cout << "findPath called, start node f cost: " << startNode.f << std::endl;
}
```

```
Before:
```

```
Initial Grid:  
S . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . G
```

```
Grid after setting obstacle:  
S . . . .  
# # # # #  
. # # # #  
. # # # #  
. # # # #  
# # # # # G  
findPath called, start node f cost: 10
```

```
After:
```

```
Initial Grid:  
S . . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . G
```

```
Grid after setting obstacle:  
S . . . . .  
# # # # #  
. . . . .  
. # # # #  
. . . . .  
# # # # G  
Path found!  
Total path cost (g): 20
```

```
void PathFind::findPath() {
    std::vector<Node> openList; // List of nodes to be evaluated

    std::vector<std::vector<int>> closedList;
    closedList.resize(numRows, std::vector<int>(numCols, 0)); // List of nodes already evaluated

    // Initializing the start node and add it to the open List
    Node startNode(startRow, startCol);
    startNode.h = manhattanDistance(startNode.point, { goalRow, goalCol });
    startNode.f = startNode.g + startNode.h;
    openList.push_back(startNode);

    // direction arrays for up, down, left, right
    int rowDir[] = { -1, 1, 0, 0 };
    int colDir[] = { 0, 0, -1, 1 };

    while (!openList.empty()) {

        // find the node with the lowest f cost in the open list
        int lowestF = 0;
        for (int i = 1; i < openList.size(); i++) {
            if (openList[i].f < openList[lowestF].f) {
                lowestF = i;
            }
        }

        Node current = openList[lowestF];
        openList.erase(openList.begin() + lowestF); // remove from open list
        closedList[current.point.row][current.point.col] = 1; // mark as visited

        // check if we reached the goal
        if (current.point.row == goalRow && current.point.col == goalCol) {
            std::cout << "Path found!" << std::endl;
            std::cout << "Total path cost (g): " << current.g << std::endl;
            return;
        }

        // explore neighbours
        for (int i = 0; i < 4; i++) {
            int newRow = current.point.row + rowDir[i];
            int newCol = current.point.col + colDir[i];

            // skip if out of bounds, obstacle, or already visited
            if (newRow < 0 || newRow > numRows - 1 || newCol < 0 || newCol > numCols) continue;
            if (grid[newRow][newCol] == 1) continue;
            if (closedList[newRow][newCol] == 1) continue;

            Node neighbour(newRow, newCol);
            neighbour.g = current.g + 1;
            neighbour.h = manhattanDistance(neighbour.point, { goalRow, goalCol });
            neighbour.f = neighbour.g + neighbour.h;
            openList.push_back(neighbour);
        }
    }

    std::cout << "No path found!" << std::endl;
}
```

```
std::vector<Node> openList; // List of nodes to be evaluated

std::vector<std::vector<int>> closedList;
closedList.resize(numRows, std::vector<int>(numCols, 0)); // List of nodes already evaluated
```

```
Node startNode(startRow, startCol);
startNode.h = manhattanDistance(startNode.point, { goalRow, goalCol });
startNode.f = startNode.g + startNode.h;
openList.push_back(startNode);
```

This creates a start node, calculates the h cost to the goal, sets f then throws it into the open list to kick things off

Quick Notes Page 11

```
// direction arrays for up, down, left, right
int rowDir[] = { -1, 1, 0, 0 };
int colDir[] = { 0, 0, -1, 1 };
```

These pair up to represent up down left and right
 So rowDir[0], colDir[0] is (-1,0) which goes up
 So rowDir[2], colDir[2] is (0,-1) which goes left

```
while (!openList.empty()) {
```

This is the main while loop which keeps going as long as there are nodes left to explore.
 If it empties without finding the goal, there is no path

```
int lowestF = 0;
for (int i = 1; i < openList.size(); i++) {
    if (openList[i].f < openList[lowestF].f) {
        lowestF = i;
    }
}
```

This finds the lowest F score.

Critical for A*

Always picks the node with the lowest f score next.

loops through the open list and tracks indexes of the cheapest node

```
Node current = openList[lowestF];
openList.erase(openList.begin() + lowestF); // remove from open list
closedList[current.point.row][current.point.col] = 1; // mark as visited
```

This grabs the cheapest node removes it from the open list and marks it as visited so that we never revisit it again, processes the current node

```
// explore neighbours
for (int i = 0; i < 4; i++) {
    int newRow = current.point.row + rowDir[i];
    int newCol = current.point.col + colDir[i];

    // skip if out of bounds, obstacle, or already visited
    if (newRow < 0 || newRow >= numRows || newCol < 0 || newCol >= numCols) continue;
    if (grid[newRow][newCol] == 1) continue;
    if (closedList[newRow][newCol] == 1) continue;
```

This checks all 4 neighbour cells.

Skips anything out of bounds, any obstacle and anything it has already visited

4 neighbours as the grid is compass natured

```
Node neighbour(newRow, newCol);
neighbour.g = current.g + 1;
neighbour.h = manhattanDistance(neighbour.point, { goalRow, goalCol });
neighbour.f = neighbour.g + neighbour.h;
openList.push_back(neighbour);
```

For each valid neighbour, g is the current nodes g plus 1 step

H is the manhattan distance to the goal

F is g+h

Then it gets pushed onto the open list to be explored later

Essentially the whole system is explore the most promising node, skip anything you've already seen, until you find the goal or run out of options

I realised a problem

In my code nodes are stored by value in std::vector<Node> so when we do openList.push_back(neighbour) it makes a copy of the node

So then when we try to set the neighbour.parent = ¤t, that pointer would make a copy that could move around in memory or be destroyed which is unreliable

can you send me obstacles that will make it not 10 just to see

Try this — it forces a long detour around a near-complete wall:

```
cpp
pf.setObstacle(0, 3);
pf.setObstacle(1, 3);
pf.setObstacle(2, 3);
pf.setObstacle(3, 3);
pf.setObstacle(4, 3);
pf.setObstacle(5, 3);
```

This is a full vertical wall cutting the grid in half with no gap, so if you keep start at (0,0) and goal at (5,5) it should print "No path found!" since there's literally no way through!

Then try this one which has a gap at the bottom forcing a long way around:

```
cpp
pf.setObstacle(0, 3);
pf.setObstacle(1, 3);
pf.setObstacle(2, 3);
pf.setObstacle(3, 3);
pf.setObstacle(4, 3);
```

This leaves a gap at row 5 so it has to go all the way down then across — you should get a g cost higher than 10!

add a bunch of zig zags so it's like a lot of steps to get to the goal because you keep giving me obstacles that make it always 10 or non-existent

Try this — a proper zigzag maze:

```
cpp
pf.setObstacle(1, 0);
pf.setObstacle(1, 1);
pf.setObstacle(1, 2);
pf.setObstacle(1, 3);
pf.setObstacle(1, 4);
pf.setObstacle(3, 1);
pf.setObstacle(3, 2);
pf.setObstacle(3, 3);
pf.setObstacle(3, 4);
pf.setObstacle(3, 5);
```

<4 0 2>

```
void Pathfind::findPath() {
    std::vector<Node*> openList;
    std::vector<std::vector<int>> closedList;
    closedList.resize(numRows, std::vector<int>(numCols, 0));

    Node* startNode = new Node(startRow, startCol);
    startNode->h = manhattanDistance(startNode->point, { goalRow, goalCol });
    startNode->f = startNode->g + startNode->h;
    openList.push_back(startNode);

    int rowDir[] = {-1, 1, 0, 0};
    int colDir[] = {0, 0, -1, 1};

    while (!openList.empty()) {
        // find lowest f cost node
        int lowestF = 0;
        for (int i = 1; i < openList.size(); i++) {
            if (openList[i]->f < openList[lowestF]->f) {
                lowestF = i;
            }
        }

        Node* current = openList[lowestF];
        openList.erase(openList.begin() + lowestF);
        closedList[current->point.row][current->point.col] = 1;

        // goal reached
        if (current->point.row == goalRow && current->point.col == goalCol) {
            std::cout << "Path found! Total steps: " << current->g << std::endl;

            // mark path on grid
            Node* trace = current;
            while (trace != nullptr) {
                if (!(trace->point.row == startRow && trace->point.col == startCol) &&
                    !(trace->point.row == goalRow && trace->point.col == goalCol)) {
                    grid[trace->point.row][trace->point.col] = 2; // mark as path
                }
                trace = trace->parent;
            }
            printPathGrid();
            return;
        }

        // explore neighbours
        for (int i = 0; i < 4; i++) {
            int newRow = current->point.row + rowDir[i];
            int newCol = current->point.col + colDir[i];

            if (newRow < 0 || newRow == numRows || newCol < 0 || newCol == numCols) continue;
            if (grid[newRow][newCol] == 1) continue;
            if (closedList[newRow][newCol] == 1) continue;

            Node* neighbour = new Node(newRow, newCol);
            neighbour->g = current->g + 1;
            neighbour->h = manhattanDistance(neighbour->point, { goalRow, goalCol });
            neighbour->f = neighbour->g + neighbour->h;
            neighbour->parent = current; // track parent!
            openList.push_back(neighbour);
        }
    }

    std::cout << "No path found!" << std::endl;
}
```



New code

```
std::vector<Node*> openList;
std::vector<std::vector<int>> closedList;
closedList.resize(numRows, std::vector<int>(numCols, 0));

Node* startNode = new Node(startRow, startCol);
startNode->h = manhattanDistance(startNode->point, { goalRow, goalCol });
startNode->f = startNode->g + startNode->h;
openList.push_back(startNode);
```

The nodes now live on the heap at fixed addresses instead of being copied around

```
Node* current = openList[lowestF];
openList.erase(openList.begin() + lowestF);
closedList[current->point.row][current->point.col] = 1;
```

New code just grabs the addresses with Node* so there is no copying shenanigans

```
Node* trace = current;
while (trace != nullptr) {
    if (!(trace->point.row == startRow && trace->point.col == startCol) &&
        !(trace->point.row == goalRow && trace->point.col == goalCol)) {
        grid[trace->point.row][trace->point.col] = 2; // mark as path
    }
    trace = trace->parent;
}
printPathGrid();
```

Old code printed and returned.

New code follows the parent pointers all the way back to the start marking each cell as a 2, then prints the visual grid

```
Node* neighbour = new Node(newRow, newCol);
neighbour->g = current->g + 1;
neighbour->h = manhattanDistance(neighbour->point, { goalRow, goalCol });
neighbour->f = neighbour->g + neighbour->h;
neighbour->parent = current; // track parent!
openList.push_back(neighbour);
```

Neighbour -> parent = current is what makes it all work now

Old code couldn't do this at a reliable standard because nodes were copies being moved around in memory.

Now each node has fixed addresses so parent pointers stay valid all the way through to the reconstruction of the grid

Need to migrate from old C pluh pluh practices to new ones

[Smart pointers \(Modern C++\) | Microsoft Learn](#)

Startign with smart pointers

The thing with smart pointers is you declare as an automatic (local) vairable - auto

Don't use new or malloc expression

Example:

In pathfinder.h

```
std::shared_ptr<Node> parent; // Pointer to parent node for path reconstruction
```

Old parent was a raw pointer that could point to freed memory. The new one is now a shared pointer so it stays valid as long as anything is referencing that node

```
auto startNode = std::make_shared<Node>(startRow, startCol);
```

New manually allocates memory that we have to manually delete. The make_shared handles all of that automatically, so when a node is no longer needed, it cleans itself up

```
std::vector<std::shared_ptr<Node>> openList;
```

The old way stored raw memory addresses. This new way stores smart pointers that track ownership and lifetime automatically

brian michelle emer surbhi natasha rachel cillian george mairtin fergus

George
David
Rachel
ben
paul
mairtin
natasha
fergus
Michelle
Emer
Surbhi
Brian

```
std::priority_queue<std::shared_ptr<Node>,
std::vector<std::shared_ptr<Node>>, CompareNode> openList;
```

Explain this code

Sure! It has three parts separated by commas inside the angle brackets:

```
cpp
std::priority_queue< std::shared_ptr<Node>, std::vector<std::shared_ptr<Node>>, Comp
```

Part 1 — `std::shared_ptr<Node>` The type of element being stored. In this case smart pointers to Nodes.

Part 2 — `std::vector<std::shared_ptr<Node>>` The underlying container the priority queue uses internally to store its elements. Priority queue is built on top of a vector under the hood.

Part 3 — `CompareNode` The comparator that tells the queue how to order elements. This is your struct:

```
cpp
struct CompareNode {
    bool operator()(const std::shared_ptr<Node>& a, const std::shared_ptr<Node>& b)
    {
        return a->f > b->f; // lowest f gets priority
    }
};
```

The `>` means lowest f floats to the top. If you flipped it to `<` it would give you the highest f first instead.

So in plain English — "a priority queue that stores smart pointers to Nodes, backed by a vector, ordered so the node with the lowest f cost is always at the top."

The rubric explicitly mentions

/ Use the C++ Standard Library
Leverage STL containers (std::vector, std::priority_queue), algorithms, and iterators where suitable.

Priority_queue which I can implement to replace the manual lowest f loop

cplusplus.com/reference/queue/priority_queue/

Got rid of a whole for loop down the bottom of this due to priority queue

```
cpp
std::priority_queue<std::shared_ptr<Node>, std::vector<std::shared_ptr<Node>>, CompareNode> openList;
std::vector<std::vector<int>> closedList;
closedList.resize(numRows, std::vector<int>(numCols, 0));

auto startNode = std::make_shared<Node>(startRow, startCol);
startNode->h = manhattanDistance(startNode->point, { goalRow, goalCol });
startNode->f = startNode->g + startNode->h;
openList.push(startNode);

int rowDir[] = { -1, 1, 0, 0 };
int colDir[] = { 0, 0, -1, 1 };

while (!openList.empty()) {
    // find lowest f cost node, priority queue handles ordering automatically
    auto current = openList.top();
    openList.pop();
```

`std::priority_queue`

This is an STL container that always keeps the most important element at the top and is ready to instantly grab it

Unlike a regular vector where you have to search through everything to find the best element, a priority queue automatically maintains the order as you push things in

So instead of the old code which had a huge for loop through every node to find the lowest f, priority queue finds the lowest f node sitting at the top

That's the main advantage of priority_queue.

Instead of looping through every node to find the lowest f value at every iteration, it keeps them sorted automatically so `top()` always gives the cheapest node instantly

In pathfinder.h

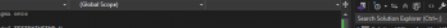
The `>` means lowest f floats at the top, if we flipped it it would give the highest

```
cpp
struct CompareNode {
    bool operator()(const std::shared_ptr<Node>& a, const std::shared_ptr<Node>& b) {
        return a->f > b->f; // higher f cost has lower priority
    }
};
```

Thursday 19 February 2026 08:49

[illegible]

Gonna copy the same structure as michelles slimprogrammer project



The screenshot shows the Visual Studio Code editor with the 'main.cpp' file open. The code in the file is as follows:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!";
    return 0;
}
```

The 'Solution Explorer' on the right side of the window shows the project structure. It includes a folder named 'main' which contains the following files:

- main.cpp
- main.o

The 'main.cpp' file is highlighted in the 'Solution Explorer'.

[illegible]

```

int main(int argc, char* argv) {
    try {
        TestFind(argv, argc);
    } catch (std::runtime_error & e) {
        std::cout << std::endl << e.what() << std::endl;
        return 1;
    }
    return 0;
}

```

```

$ g++ -std=c++11 test.cpp
$ ./testFind.cpp
1

```

So I could in the future, once I add different heuristics, run it like `pathfinder.exe manhattan`, or `pathfinder.exe euclidean`.

[illegible][illegible][illegible]

```

// ...
Michelle Lynch
//
#include <iostream> // cin, endl
#include <string> // str, string
#include <string.h> // strlen
#include <math.h> // sqrt, pow, etc.
// ...

int main(int argc, char* argv)
{
    try {
        Michelle Lynch;
    }
    catch (...) {
        cout << "Error: " << argv[1] << endl;
        return 1;
    }
    return 0;
}

```

```
#include <iostream>
using namespace std;

// Global variable
int x = 0;

// Function prototype
void TestFunction(int arg);
void TestCopyCat();
void TestStaticDataMember();

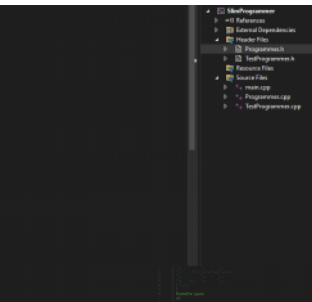
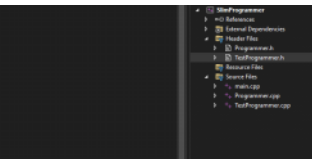
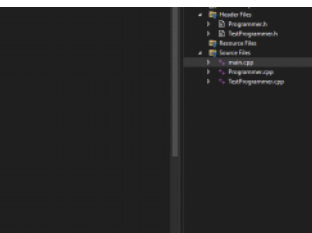
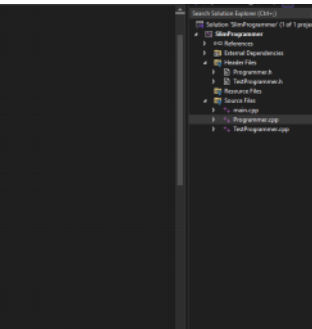
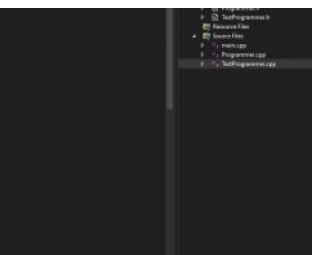
// Main function
int main() {
    // Call TestFunction
    TestFunction(10);

    // Call TestCopyCat
    TestCopyCat();

    // Call TestStaticDataMember
    TestStaticDataMember();

    return 0;
}
```

[illegible]



ok i created all those tests now. what do you think we would be on in the rubric now?

Honestly you're probably sitting around 65-70 now. Here's where you stand:

What you've added that's useful:

- Smart pointers [\[1\]](#)
- Priority queue [\[2\]](#)
- Test structure matching lecturer's style [\[3\]](#)
- Edge cases covered [\[4\]](#)
- Good class structure throughout [\[5\]](#)
- Comments [\[6\]](#)

What's still missing for 70+:

- Euclidean distance second heuristic -- rubric mentions it explicitly
- Range-based for loops -- you still have: `for (int i = 0; i < G; i++) InFirstPath`
- Coordinate path printing -- `(G, G) <- (2, 45, ...)`
- The report -- this is probably the biggest gap right now

To be read with you -- the report could be worth as much as the code itself based on how the rubric reads. It specifically wants design decisions explained, limitations discussed, and a reflective section. The journey you've taken with this code -- raw pointers to smart pointers, manual search to priority queue, single file to modular structure -- is genuinely good material to write about.

If you tackle Euclidean distance and range-based loops quickly since they're small changes, then seriously start the report. Want to do those two first?

A big issue I'm delaying right now is that I am currently still manually assigning the obstacles etc so I need a way to randomise it

But that's an issue for another day, I don't think that will boost my grade a whole lot. Looking at the rubric:

Implement A* Algorithm
• Use program design
• Represent a problem graph with obstacles.
• Implement heuristic based pathfinding (e.g. Manhattan or Euclidean distance).
• Output the shortest path from start to goal as a color format.

I wanna implement euclidean distance right now. After that I'll add in chebyshev

Euclidean Distance: $(\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2})$.

So let a = row
r = col
We get point 1 * row - point 2 * row + a
Point 1 * col - point 2 * col + b

Then square root (a * a + b * b)

```
double euclideanDist(int row1, int col1, int row2, int col2) {
    int a = row1 - row2;
    int b = col1 - col2;
    int result = a * a + b * b;
    return sqrt(result);
}
```

I'm at a cross roads right now, I can either make it so the user manually inputs select manhattan or euclidean.

Or, I can add an argument pass thing and realise that so we can pass it manhattan or euclidean in the function call

From research I can use enum class

Enum Classes C++ and Their Advantage over Enum Data Type

It's good practice for modern C++ too

Enum Classes in C++ and Their Advantage over Enum Data Type

Last updated: 11 Jul, 2025

Enums, or enumerated types, are user-defined data types in C++ that allow the programmer to define a set of named values. These values are fixed and predetermined at the time of declaration. However, traditional enums have some limitations, which have been addressed by the introduction of Enum Classes in C++11.

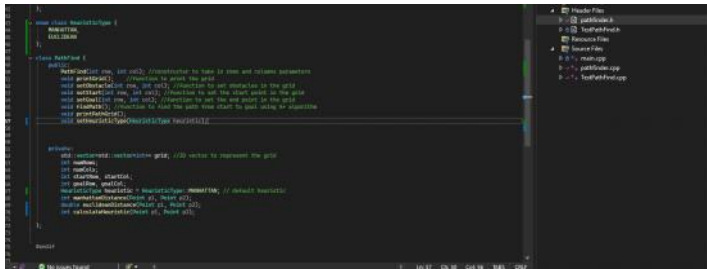
I can declare something like

```
int main()
{
    enum class Color { Red, Green, Blue };
    enum class Color2 { Red, Black, White };
    enum class People { Good, Bad };
}
```

Enum class Heuristic (Manhattan, Euclidean, Chebyshev when I add it)

And then instantiate it like `g2.setHeuristicToChebyshev()` or `Heuristic::Manhattan`

// instantiating the enum class
Color x = Color::Green;



I created a function that lets you specify the Heuristic Type you are using (Manhattan, Euclidean)

The default heuristic is Manhattan

I also then made a private one which calculates the h value based on the heuristic you choose

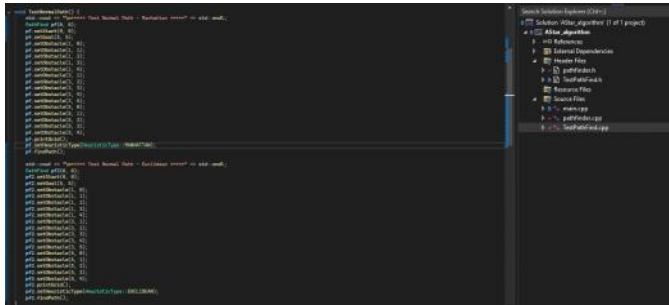
```
void Heuristic::setHeuristicType(HeuristicType heuristicType) {
    heuristicType = heuristicType;
}

int Heuristic::calculateManhattanDist() const {
    return rowDiff + colDiff;
}

int Heuristic::calculateEuclideanDist() const {
    return sqrt(rowDiff * rowDiff + colDiff * colDiff);
}

int Heuristic::calculateChebyshevDist() const {
    return max(rowDiff, colDiff);
}
```

In here, if the method called is specified as Manhattan it will return the else. If it's euclidean it calculates and type casts since it uses the square root function. Then to test it we call it as follows:



Both give the same value as the g2 in the same with a fixed h way path



Interestingly,

If we remove all the obstacles, we can see the paths the different heuristics take are different

heuristic
They look different but cost the same

They differ in how they estimate costs, but the movement rules don't matter because its only 4 directional movement

- Manhattan counts row + col differences — accurate for 4 directions
- Euclidean measures straight line distance — slightly underestimates for 4 directions
- Chebyshev takes max of row/col — designed for 8 directional movement so it underestimates here

can also create a test for me that compares the 2 different heuristics I have mentioned, as follows: [chrisphew](#) vs [one go](#)

```

def testMethod2(a):
    """Test method 2"""
    # Create a list of 10 random numbers
    randomNumbers = [random.random() * 100 for _ in range(10)]

    # Sort the list
    randomNumbers.sort()

    # Print the sorted list
    print("Sorted list: ", randomNumbers)

    # Return the sorted list
    return randomNumbers

# Test method 2
testMethod2()

```

I think the only fixes I have left out are to print out the path as coordinates as well, and making the code more modular?

- Make heuristics multilevel refactor
- Coordinate path printing
- Randomise the obstacles and grid and stuff
- Handle same start and goal edge case properly

23/02

Monday 23 February 2026 13:10

Summary of last weeks work

18/02 Session

- Implemented the core A* algorithm — the `findPath()` function. This included setting up the open and closed lists, the main while loop that explores nodes by lowest F score, and neighbour expansion with bounds/obstacle checking using Manhattan distance as the heuristic.
- Hit a memory bug: nodes stored by value in `std::vector<Node>` meant parent pointers became unreliable when the vector moved things in memory. Fixed this by moving nodes to the heap using raw pointers (`Node*`), giving each node a fixed address so parent pointer tracking worked correctly for path reconstruction.
- Upgraded to **smart pointers** (`std::shared_ptr`, `std::make_shared`) to replace the raw pointer approach — no more manual memory management, automatic cleanup.
- Replaced the manual "find lowest F" for-loop with `std::priority_queue`, which keeps the cheapest node at the top automatically — cleaner and more efficient.

19/02 Session

- Set up a proper **test project structure** with `testPathFinder.cpp/.h`, separating tests from main code. Tests cover: normal path, no path found, 1x1 grid, and `start == goal`.
- Added `argc/argv` support so heuristics could eventually be passed as command-line arguments.
- Implemented **Euclidean distance** heuristic and refactored heuristic selection using an **enum class** (`HeuristicType::MANHATTAN`, `EUCLIDEAN`, `CHEBYSHEV`) — good modern C++ practice.
- Added **Chebyshev distance** heuristic.
- Switched neighbour exploration to use **range-based for loops** with a vector of direction pairs.
- Observed that all three heuristics give the same step count on a 4-directional grid, and noted why (Manhattan is exact, Euclidean/Chebyshev underestimate for 4-directional movement).

Planned next steps:

- Move heuristics into a separate file (modular refactor)
- Coordinate path printing
- Randomise obstacle/grid generation
- Add `_` to stuff
- Do the matrix lab on it
- Friend function maybe
- Add pdf output
- Operator overloading
- Composition over inheritance (talk about that)
- DIMENSIONS TRY CHECK ALL THAT

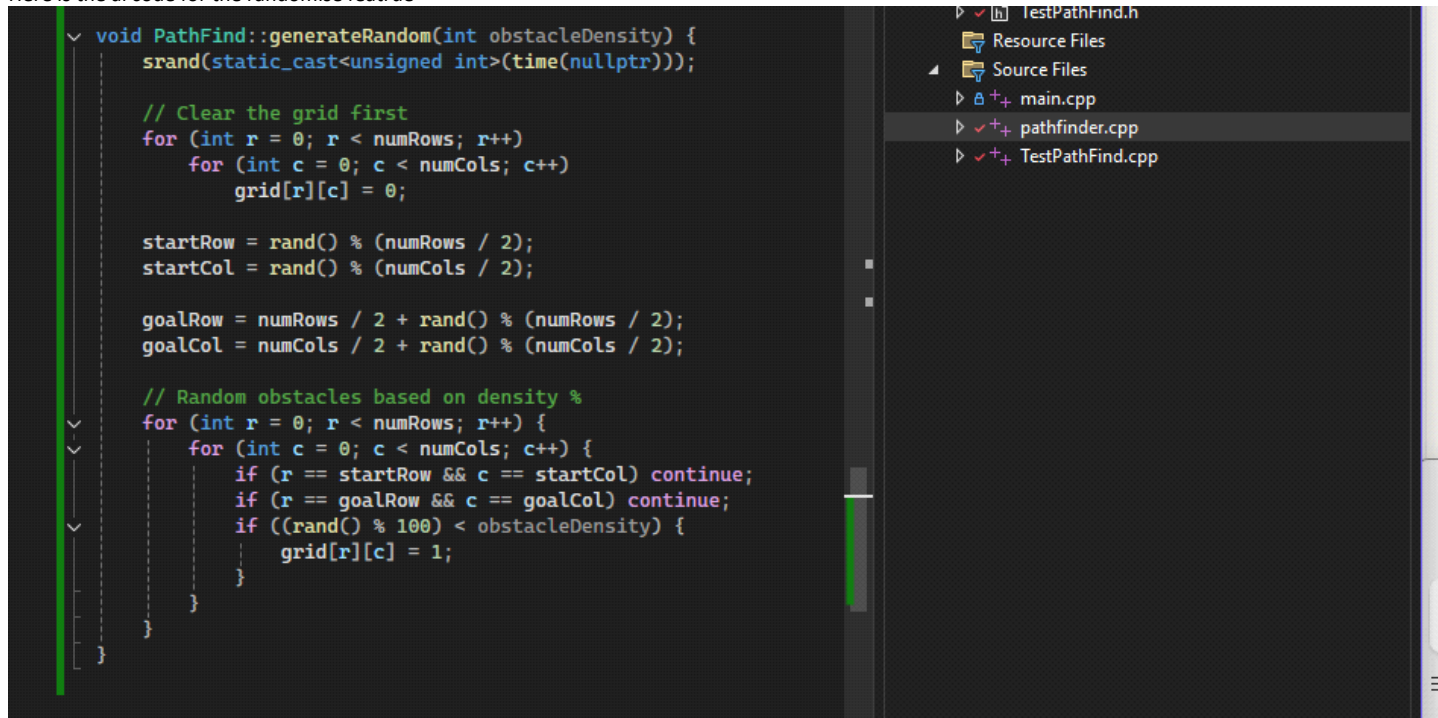
12/03

Thursday 12 March 2026 12:10

Planned next steps:

- Move heuristics into a separate file (modular refactor)
- Add more classes
- Coordinate path printing
- Randomise obstacle/grid generation (done by ai, but it is very bad modern c++ practices
le r++ instead of preincrementing, and old style loops and the rand is not a c++ one
rather c style code)
- Add _ to stuff like rows and cols
- Do the matrix lab on it
- Friend function maybe
- Operator overloading
- Composition over inheritance (talk about that in github pages report)
- DIMENSIONS TRY CHECK ALL THAT
- Add try catch

Here is the ai code for the randomise feature



```
void PathFind::generateRandom(int obstacleDensity) {
    srand(static_cast<unsigned int>(time(nullptr)));

    // Clear the grid first
    for (int r = 0; r < numRows; r++)
        for (int c = 0; c < numCols; c++)
            grid[r][c] = 0;

    startRow = rand() % (numRows / 2);
    startCol = rand() % (numCols / 2);

    goalRow = numRows / 2 + rand() % (numRows / 2);
    goalCol = numCols / 2 + rand() % (numCols / 2);

    // Random obstacles based on density %
    for (int r = 0; r < numRows; r++) {
        for (int c = 0; c < numCols; c++) {
            if (r == startRow && c == startCol) continue;
            if (r == goalRow && c == goalCol) continue;
            if ((rand() % 100) < obstacleDensity) {
                grid[r][c] = 1;
            }
        }
    }
}
```

Very bad c++ practices r++

Old style loops

C rand function

It works however

***** Test Random Path *****

```
. . . . .
. . . . .
. S . . # . . . .
. . . . . # # .
. . . . . # #
. . . . # . . . .
# . . . . . . . .
. . . . . . . . .
. . . . . # # .
. . . . . G . . .
```

Path Grid:

```
. . . . .
. . . . .
. S . . # . . . .
. * . . . # . # .
. * * . . . # #
. . * * * # . . .
# . . . * . . . .
. . . . * . . . .
. . . . * * . # # .
. . . . * G . . .
```

Path found! Total steps: 12

16/03

Monday 16 March 2026 14:33

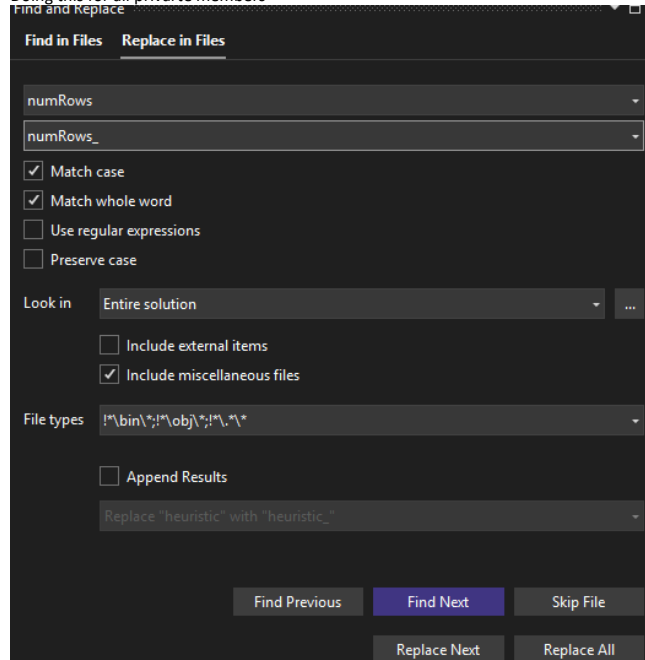
Gonna change all the private members to have trailing underscores
This is so you can tell at a glance whether a variable is a member or local
Gonna do this by ctrl f and replacing exact cases

Changed heuristic

```
void PathFind::setHeuristicType(HeuristicType heuristic_) {
    this->heuristic_ = heuristic_;
}

int PathFind::calculateHeuristic(Point p1, Point p2) {
    if (heuristic_ == HeuristicType::EUCLIDEAN) {
        return (int)euclideanDistance(p1, p2);
    }
    else if (heuristic_ == HeuristicType::CHEBYSHEV) {
        return chebyshevDistance(p1, p2);
    }
    else {
        return manhattanDistance(p1, p2);
    }
}
```

Doing this for all private members



```
private:
    std::vector<std::vector<int>> grid; //2D vector to represent the grid
    int numRows_;
    int numCols_;
    int startRow_, startCol_;
    int goalRow_, goalCol_;
```

Did that

In one of the labs for exception handling we did an out of bounds check so I wanna replace my else statements with that

```
void PathFind::setObstacle(int row, int col) { //function to set obstacles in the grid
    if (row >= 0 && row < numRows_ && col >= 0 && col < numCols_) { //checking if the row and col are within bounds
        grid[row][col] = 1; //setting the obstacle at specified position
    }
    else {
        std::cout << "Error: Out of bounds" << std::endl;
    }
}

void PathFind::setStart(int row, int col) { //function to set the start point in the grid
    if (row >= 0 && row < numRows_ && col >= 0 && col < numCols_) { //checking if the row and col are within bounds
        startRow_ = row; //setting the start row
        startCol_ = col; //setting the start column
    }
    else {
        std::cout << "Error: Start Out of bounds" << std::endl;
    }
}

void PathFind::setGoal(int row, int col) { //function to set the end point in the grid
    if (row >= 0 && row < numRows_ && col >= 0 && col < numCols_) { //checking if the row and col are within bounds
        goalRow_ = row; //setting the goal row
        goalCol_ = col; //setting the goal column
    }
    else {
        std::cout << "Error: Goal Out of bounds" << std::endl;
    }
}
```

```
***** Test Normal Path - Manhattan *****
```

```
SetObstacle: row out of bounds
```

```
void PathFind::setObstacle(int row, int col) { //function to set obstacles in the grid
    if (row >= 0 && row < numRows_ && col >= 0 && col < numCols_) { //checking if the row and column
        grid[row][col] = 1; //setting the obstacle at specified position
    }
    else {
        throw std::out_of_range("setObstacle: row out of bounds"); //throwing an error if the position
    }
}

void PathFind::setStart(int row, int col) { //function to set the start point in the grid
    if (row >= 0 && row < numRows_ && col >= 0 && col < numCols_) { //checking if the row and column
        startRow_ = row; //setting the start row
        startCol_ = col; //setting the start column
    }
    else {
        throw std::out_of_range("setStart: row out of bounds"); //throwing an error if the position
    }
}

void PathFind::setGoal(int row, int col) { //function to set the end point in the grid
    if (row >= 0 && row < numRows_ && col >= 0 && col < numCols_) { //checking if the row and column
        goalRow_ = row; //setting the goal row
        goalCol_ = col; //setting the goal column
    }
    else {
        throw std::out_of_range("SetGoal: row out of bounds"); //throwing an error if the position
    }
}
```

Did this because it's a quick change and fix

It also completes the try catch in mainn.cpp

Correct c++ and used for exception handling allowed us to gracefully handle the error that occurred during program execution

Next thing I want to make heuristics more modular and into classes

Why? Because it's all in pathfind and it's doing 2 unrelated things

Calculating distance and pathfinding logic

So splitting these into their respective responsibilities is much better

Moved these commented ones to heuristics.h

```
private:
    std::vector<std::vector<int>> grid_; //2D vector to represent the grid_
    int numRows_;
    int numCols_;
    int startRow_, startCol_;
    int goalRow_, goalCol_;
    HeuristicType heuristic_ = HeuristicType::MANHATTAN; // default heuristic_
    //int manhattanDistance(Point p1, Point p2);
    //double euclideanDistance(Point p1, Point p2);
    //int chebyshevDistance(Point p1, Point p2);
    int calculateHeuristic(Point p1, Point p2);
};
```

Removed Point and HeuristicType from pathfinder.h and added them to a file called types.h

What I did

- types.h now owns Point and HeuristicType — shared neutral header, no circular dependency possible
- pathfinder.h and heuristics.h both include types.h only
- Heuristics methods are static — no instantiation needed, pure utility class
- calculateHeuristic is now a single delegation call — PathFind no longer knows *how* distances are calculated, only *that* they are

Next I am changing all my post increments to pre increments

Just did that

Need to do coordinate path printing

```
***** Test Normal Path - Euclidean *****
```

```
S . . . . .
# # # # .
. . . . .
. # # # #
. . . . .
# # # # G
```

```
Path: (0, 0) -> (0, 1) -> (0, 2) -> (0, 3) -> (0, 4) -> (0, 5) -> (1, 5) -> (2, 5) -> (2, 4) -> (2, 3) -> (2, 2) -> (2, 1) -> (2, 0) -> (3, 0) -> (4, 0) -> (4, 1) -> (4, 2) -> (4, 3) -> (4, 4) -> (4, 5) -> (5, 5) -> Path found! Total steps: 20
```

```
C:\Users\G00419108@atu.ie\OneDrive - Atlantic TU\YEAR 4\sem2\C++\AStar_algorithm\Debug\AStar_algorithm.exe (process 17328) exited with code 0 (0x0).
```

Generate random needs to change now

```
void PathFind::generateRandom(int obstacleDensity) {
    srand(static_cast<unsigned int>(time(nullptr)));

    // Clear the grid first
    for (int r = 0; r < numRows_; ++r)
        for (int c = 0; c < numCols_; ++c)
            grid_[r][c] = 0;

    startRow_ = rand() % (numRows_ / 2);
    startCol_ = rand() % (numCols_ / 2);

    goalRow_ = numRows_ / 2 + rand() % (numRows_ / 2);
    goalCol_ = numCols_ / 2 + rand() % (numCols_ / 2);

    // Random obstacles based on density %
    for (int r = 0; r < numRows_; ++r) {
        for (int c = 0; c < numCols_; ++c) {
            if (r == startRow_ && c == startCol_) continue;
            if (r == goalRow_ && c == goalCol_) continue;
            if ((rand() % 100) < obstacleDensity) {
                grid_[r][c] = 1;
            }
        }
    }
}
```

Instead of it randomising the start and end its just gonna do obstacles

Its not printing the path on the grid tho

```
void TestRandomPath() {
    std::cout << "\n***** Test Random Path *****" << std::endl;
    PathFind pf(10, 10);
    pf.setStart(0, 0);
    pf.setGoal(9, 9);
    pf.generateRandom(25); //obstacle density
    pf.printGrid();
    pf.setHeuristicType(HeuristicType::MANHATTAN);
    pf.findPath();

    void TestHeuristicComparison() {
        std::cout << "\n***** Heuristic Comparison *****" << std::endl;
        auto runWithHeuristic = [](HeuristicType type) {
            PathFind pf(6, 6);
            pf.setStart(0, 0);
            pf.setGoal(5, 5);
            pf.setObstacle(1, 0);
            pf.setObstacle(1, 4);
            pf.setObstacle(3, 1);
            pf.setObstacle(3, 2);
            pf.setObstacle(3, 4);
            pf.setObstacle(3, 5);
            pf.setObstacle(5, 0);
            pf.setObstacle(5, 2);
            pf.setObstacle(5, 3);
            pf.setHeuristicType(type);
            std::cout << "\n----- " << name << " -----" << std::endl;
        };
    }
}
```

```
***** Test Random Path *****
S . . . . # . . .
. . . # . # . .
# . . . . . . .
. . # . . . . #
. . # . . # . .
. . . . # . . .
. . . # . # . .
. # . # . . # .
. . # . . . . .
. . . . . # . . G

Path: (0, 0) -> (0, 1) -> (1, 1) -> (1, 2) -> (2, 2) -> (2, 3) -> (2, 4) -> (2, 5) -> (3, 5) -> (4, 5) -> (4, 8) -> (5, 8) -> (6, 8) -> (6, 7) -> (7, 7) -> (8, 7) -> (8, 8)
20

C:\Users\G00419108@atu.ie\OneDrive - Atlantic TU\YEAR 4\sem2\C++\C++\ASTar\src\main.cpp (12108) exited with code 0 (0x0).
Press any key to close this window . . .
```

That's a bug

Added timing tests fixed that error aswell

Thursday 19 March 2026 09:09

```
friend for point
struct Point {
    int row;
    int col;

    friend std::ostream& operator<<(std::ostream& os, const Point& p) {
        return os << "(" << p.row << ", " << p.col << ")";
    }
};
```

[illegible]

```

Old calculateHeuristics
int PathFind::calculateHeuristic(Point p1, Point p2) {
    if (Heuristic == HeuristicType::L1_DISTANCE) {
        return (int)euclideanDistance(p1, p2);
    }
    else if (Heuristic == HeuristicType::CHEBYSHEV) {
        return chebyshevDistance(p1, p2);
    }
    else {
        return manhattanDistance(p1, p2);
    }
}

```

```
New calculate heuristics
let PathFind::calculateHeuristic(Point p1, Point p2) {
    return Heuristics::calculateHeuristic(p1, p2);
}
```

```

Heuristics.h
class Heuristics {
public:
    static int manhattan(Point p1, Point p2);
    static double euclidean(Point p1, Point p2);
    static int chebyshev(Point p1, Point p2);
    static int calculate(HeuristicType type, Point p1, Point p2);
};

```

```
// explore neighbours
for (int i = 0; i < 4; i++) {
    int newRow = current.point.row + rowDir[i];
    int newCol = current.point.col + colDir[i];

    // skip if out of bounds, obstacle, or already visited
    if (newRow < 0 || newRow >= newRows || newCol < 0 || newCol >= newCols) continue;
    if (grid[newRow][newCol] == 1) continue;
    if (closedList[newRow][newCol] == 1) continue;

    Node newNeighbour(newRow, newCol);
    newNeighbour.g = current.g + 1;
    newNeighbour.h = manhattanDistance(newNeighbour.point, { goalRow, goalCol });
    newNeighbour.f = newNeighbour.g + newNeighbour.h;
    openList.push_back(newNeighbour);
}
```

```

    explore_neighbours
    for (int i = 0; i < 4; i++) {
        int newRow = current.point.row + rowDir[i];
        int newCol = current.point.col + colDir[i];

        // skip if out of bounds, obstacle, or already visited
        if (newRow < 0 || newRow == numRows || newCol < 0 ||
            newCol == numCols == 1) continue;
        if (closedList[newRow][newCol] == 1) continue;

        Made neighbour(newRow, newCol);
        neighbour.g = current.g + 1;
        neighbour.h = manhattanDistance(neighbour.point, goal);
        neighbour.f = neighbour.g + neighbour.h;
        openList.push_back(neighbour);
    }
}

```

```

void* neighbour
{
    for (const direction directions) {
        int newrow = current->parent_row + direction.first;
        int newcol = current->parent_col + direction.second;

        if (newrow < 0 || newrow > numbers, || newcol < 0 || newcol > numbers)
            continue;
        if (grid[newrow][newcol] == 1) continue;
        if (closedList.find(newrow)[newcol] == 1) continue;

        auto neighbour = std::make_shared<Node>(newrow, newcol);
        neighbour->current = current + 1;
        neighbour->h = calculateHeuristic(neighbour->parent, grid, numbers, goal);
        neighbour->parent = neighbour->h;
        neighbour->parent = current; // track parent!
        openList.push(neighbour);
    }
}

```

Quick Notes Page 29

```
enum HandPile {
    HANNAHAN,
    EILTHEAN,
    CHETASHEV
};
```

```
enum class WeatherType {
    HANNAH_TAN,
    CLEVELAND,
    CHEBYSEV
};
```

```
Test no path
**** Test No Path ****
S # . .
d # . .
. . .
. . G
No path found!
```

```
newCol >= numCols) continue;
goalRow, goalCol ]});
```

```
1), {0, 1} }); // up, down, left, right directions
```

```

// Old random
void generateRandom() {
    srand(time(NULL));

    // Clear the grid, first
    for (int r = 0; r < numRows; ++r)
        for (int c = 0; c < numCols; ++c)
            grid[r][c] = 0;

    startRow = rand() % (numRows - 2);
    startCol = rand() % (numCols - 2);

    goalRow = numRows - 2 + rand() % (numRows - 2);
    goalCol = numCols - 2 + rand() % (numCols - 2);

    // Remove obstacles based on density %
    for (int r = 0; r < numRows; ++r) {
        for (int c = 0; c < numCols; ++c) {
            if (r == startRow || c == startCol) continue;
            if (r == goalRow || c == goalCol) continue;
            if ((rand() % 100) < obstacleDensity) {
                grid[r][c] = 1;
            }
        }
    }
}

```

```

// Old Pathfind
void generateRandom() {
    std::mt19937 rng(std::random_device{}());
    std::uniform_int_distribution<int> randInt(0, 99);

    // Clear grid using memset
    for (auto i = grid.begin(); i != grid.end(); ++i) {
        std::fill(i->begin(), i->end(), 0);
    }

    // Place obstacles by routing through setObstacle
    for (int r = 0; r < numRows; ++r) {
        for (int c = 0; c < numCols; ++c) {
            if (r == startRow || c == startCol) continue;
            if (r == goalRow || c == goalCol) continue;
            if ((randInt(rng) < obstacleDensity)) {
                setObstacle(r, c);
            }
        }
    }
}

```

